

CSC 250: Foundations of Computer Science I

Spring 2026

Chapter 3: Construction Techniques

Sakhawat Hossan

Department of Computer Science
University of North Carolina at Greensboro

Section 3.1 Induction

To understand how induction works, suppose there is a professor who brings to class a bottomless bag of assorted miniature candy bars. She offers to share the candy in the following way. First, she lines the students up in order. Next she states two rules:

- The student at the beginning of the line gets a candy bar.
- If a student gets a candy bar, then the following student in line also gets a candy bar.

Let's number the students by their order in line, starting the count with 0, as usual in computer science. Now we can understand the second rule as a short description of a whole sequence of statements:

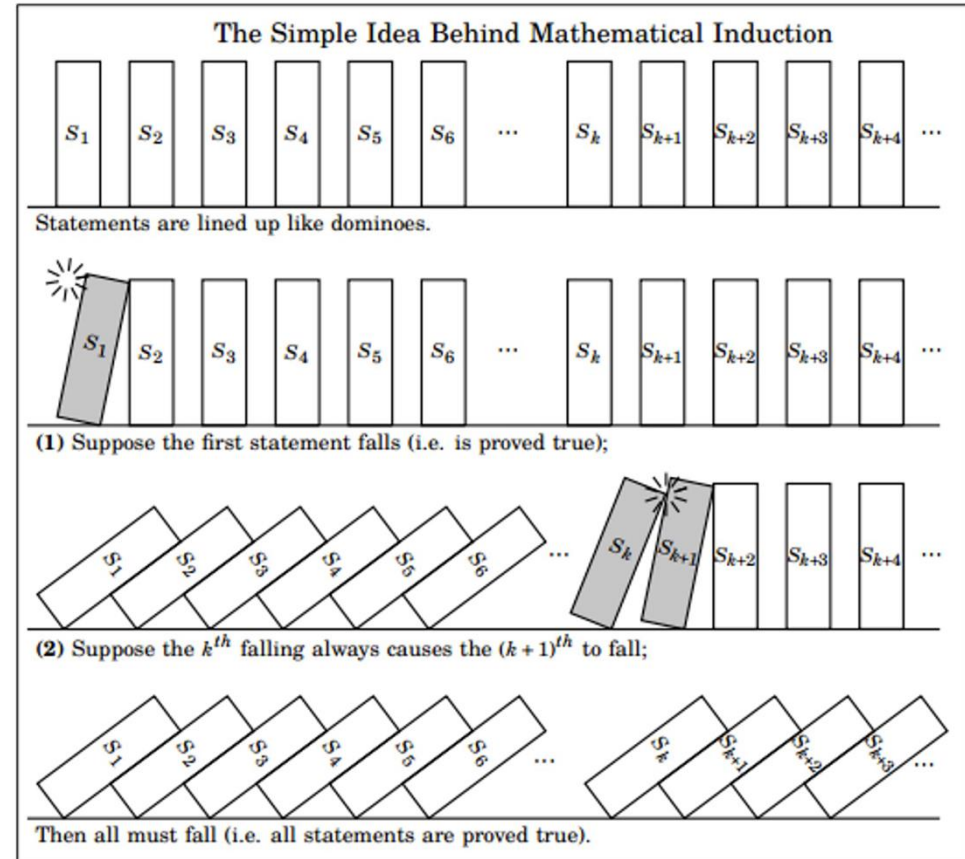
- If student 0 gets a candy bar, then student 1 also gets one.
- If student 1 gets a candy bar, then student 2 also gets one.
- If student 2 gets a candy bar, then student 3 also gets one.
- $\vdots$

Of course this sequence has a more concise mathematical description:

If student n gets a candy bar, then student $n + 1$ gets a candy bar, for all nonnegative integers n .

Induction

So, suppose you are student 17. By these rules, are you entitled to a miniature candy bar? Well, student 0 gets a candy bar by the first rule. Therefore, by the second rule, student 1 also gets one, which means student 2 gets one, which means student 3 gets one as well, and so on. By 17 applications of the professor's second rule, you get your candy bar! Of course the rules actually guarantee a candy bar to *every* student, no matter how far back in line they may be.



Proof by Mathematical Induction

Mathematical induction is a useful technique for proving that a statement $P(n)$ is true for all nonnegative integers n , or, more generally, for all integers at or above some lower limit.

Steps to perform proof by Mathematical Induction:

- **Step 1 Basis (base case):** Prove that the statement holds for the first natural number n . Usually, $n = 0$ or $n = 1$.
- **Step 2 Induction Hypothesis:** Assume that the statement is true for $n = k$
- **Step 3 Inductive Step:** If it is true for $n=k$, prove that the statement is true for $n = k+1$. Plug in $k+1$, and then simplify or rearrange RHS such that $LHS = RHS$. When both sides are equal, the inductive step is completed.
- **Step 4 Conclusion:** Invoke the Induction. By mathematical induction, it is proved that the statement is true for all nonnegative n .

Induction Example

Prove by induction that

$$1 + 2 + 3 + \cdots + n = \frac{n(n + 1)}{2}$$

for every positive integer n .

Solution

Let $P(n)$ be the statement:

$$1 + 2 + 3 + \cdots + n = \frac{n(n + 1)}{2}.$$

Since this is an equation, at each stage of the proof we will show that the left-hand side equals the right-hand side.

Base Case

When $n = 1$:

Left-hand side: 1

Induction Example

Right-hand side:

$$\frac{1(1 + 1)}{2} = \frac{2}{2} = 1$$

So $P(1)$ is true.

Induction Hypothesis

Assume that $P(k)$ is true for some positive integer k .

That is, assume:

$$1 + 2 + 3 + \cdots + k = \frac{k(k + 1)}{2}.$$

Induction Step

We will now show that $P(k + 1)$ is correct by taking the left-hand side of the equation and re-arranging it until it equals the right-hand side.

In this case, the right-hand side of the formula should be

$$\frac{(k + 1)((k + 1) + 1)}{2},$$

because we have replaced n with $(k + 1)$.

Induction Example

So starting with the left-hand side, we have:

$$\begin{aligned} 1 + 2 + 3 + \cdots + k + (k + 1) &= (1 + 2 + 3 + \cdots + k) + (k + 1) \\ &= \frac{k(k + 1)}{2} + (k + 1) \text{ (by the induction hypothesis)} \\ &= \frac{k(k + 1)}{2} + \frac{2(k + 1)}{2} \\ &= \frac{k(k + 1) + 2(k + 1)}{2} \\ &= \frac{(k + 1)(k + 2)}{2} = \frac{(k + 1)((k + 1) + 1)}{2}. \end{aligned}$$

And this is exactly the right-hand side, so $P(k + 1)$ is correct.

Conclusion By mathematical induction,

$$1 + 2 + 3 + \cdots + n = \frac{n(n + 1)}{2}$$

for all positive integers n .

Induction Example 2

Prove by induction that

$$n^3 + 2n$$

is divisible by 3 for every non-negative integer n .

Solution

Let $P(n)$ be the statement:

$$n^3 + 2n \text{ is divisible by 3.}$$

Base Case

When $n = 0$:

$$\begin{aligned} 0^3 + 2(0) &= 0 \\ 0 &= 3 \times 0 \end{aligned}$$

So $P(0)$ is true.

Induction Hypothesis

Assume $P(k)$ is true for some non-negative integer k . That means:

$$k^3 + 2k = 3m$$

for some integer m .

Induction Example 2

Induction Step

We now prove that $P(k + 1)$ is true. What we are aiming to show

For $n = k + 1$, the statement becomes:

$$(k + 1)^3 + 2(k + 1)$$

is divisible by 3. Since this is not an equation but a divisibility statement, our goal is to show:

$$(k + 1)^3 + 2(k + 1) = 3 \times (\text{some integer})$$

Left-Hand Side (LHS)

Start with the left-hand side:

$$\text{LHS} = (k + 1)^3 + 2(k + 1)$$

Expand:

$$= k^3 + 3k^2 + 3k + 1 + 2k + 2$$

Combine like terms:

$$= k^3 + 2k + 3k^2 + 3k + 3$$

Now group carefully:

$$= (k^3 + 2k) + 3(k^2 + k + 1)$$

Induction Example 2

Use the Induction Hypothesis

By the induction hypothesis, we assumed:

$$k^3 + 2k = 3m$$

for some integer m . Substitute this into the expression:

$$= 3m + 3(k^2 + k + 1)$$

Factor out 3:

$$= 3(m + k^2 + k + 1)$$

Conclusion of Induction Step

Since $m + k^2 + k + 1$ is an integer, the entire expression is a multiple of 3. Thus,

$$(k + 1)^3 + 2(k + 1)$$

is divisible by 3. Therefore, $P(k + 1)$ is true.

Hence, by mathematical induction, $n^3 + 2n$ is divisible by 3 for all non-negative integers n .

Exercises

Prove the following statements by mathematical induction.

1. Prove that for all positive integers n ,

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2 (n + 1)^2}{4}$$

2. Prove that for all non-negative integers n ,

$$9^n - 1$$

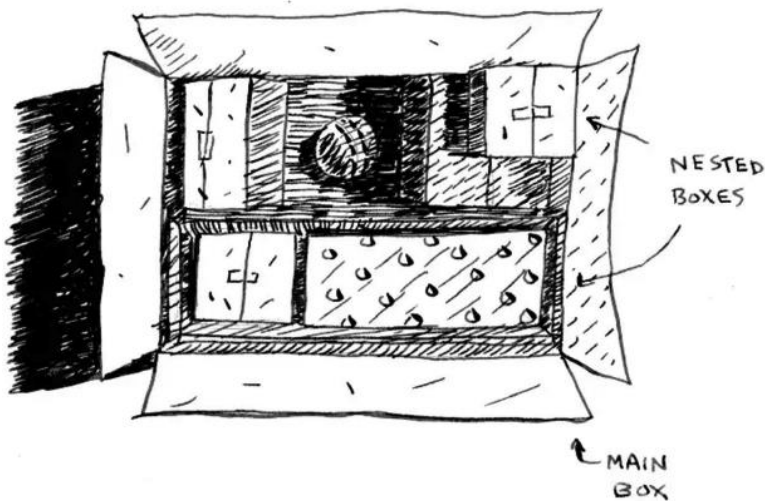
is divisible by 8.

3. Prove that for all positive integers n ,

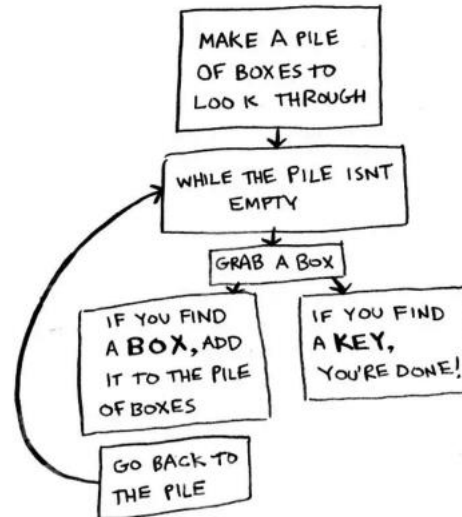
$$2^n + 1 \geq n^2.$$

Section 3.2 Recursion

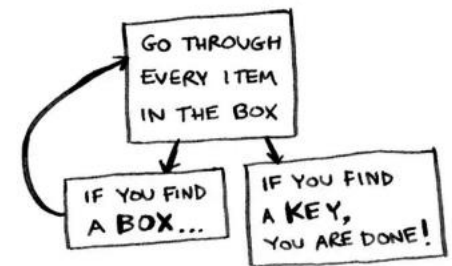
Imagine you go to open your bedroom door and it's locked. Your three-year-old son pops in from around the corner and let's you know he hid the only key in a box. You open the box only to find... More boxes. Boxes inside of boxes. And you don't know which one has the key!



Iterative Approach



Recursive Approach



Iterative vs Recursive Method

An **iterative method** is a problem-solving approach that repeats a block of instructions using control structures such as: for loops, while loops, do-while loops. It continues executing until a stopping condition is met.

A **recursive method** is a problem-solving approach in which a function calls itself to solve smaller instances of the same problem.

Example:

Iterative Factorial

```
factorial(n):  
    result = 1  
    for i = 1 to n:  
        result = result * i  
    return result
```

Recursive Factorial

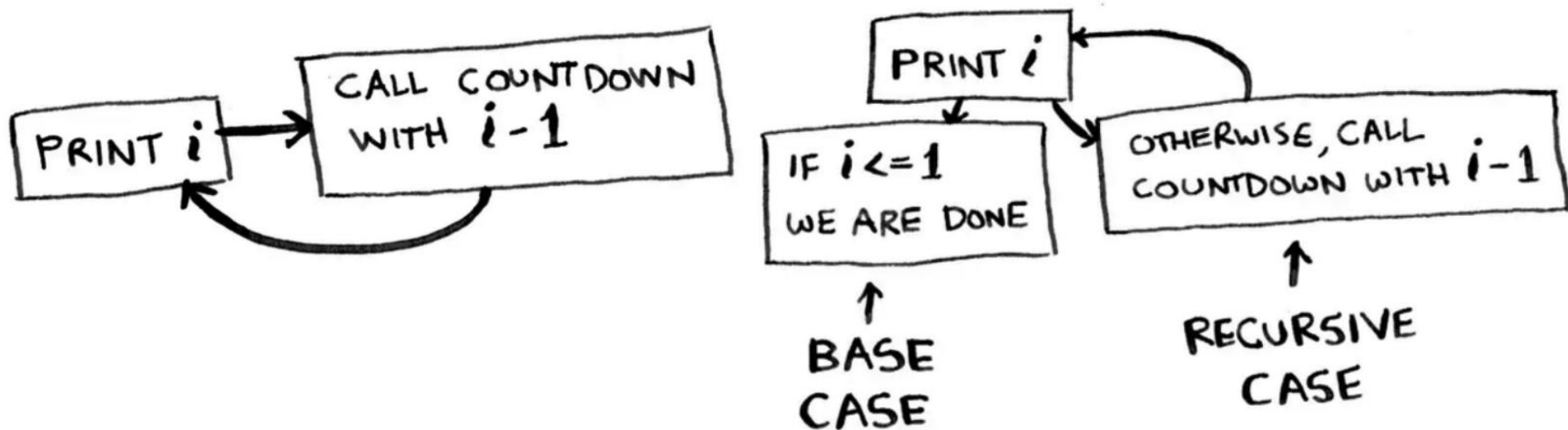
```
factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Iterative vs Recursive Method

Feature	Iterative	Recursive
Structure	Uses loops	Function calls itself
Memory Usage	Low (no extra stack)	Higher (call stack used)
Speed	Usually faster	Slightly slower (function call overhead)
Base Case Needed?	No	Yes (mandatory)
Risk	Infinite loop	Stack overflow
Code Clarity	Sometimes longer	Often cleaner & mathematical
Use Case	Performance and memory are important. The problem is simple repetition. Working in systems with limited stack space.	The problem is naturally recursive: Tree traversal, Graph DFS, Merge Sort, Quick Sort, Divide and Conquer. Mathematical definitions: Factorial, Fibonacci, Backtracking problems

Recursion

Something you have to look out for when writing a recursive function is an infinite loop. This is when the function keeps calling itself... And never stops calling itself! For instance, you may want to write a countdown function. This function will keep counting down forever.

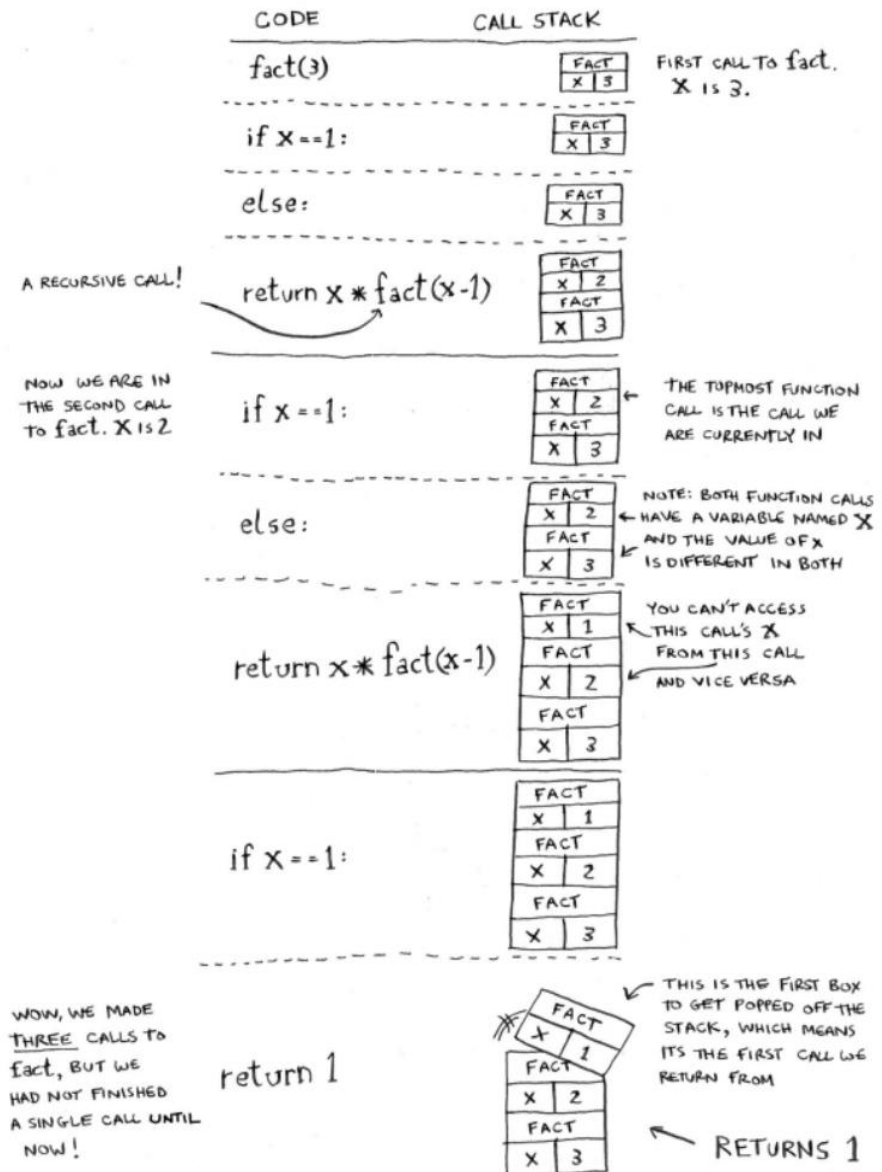


A recursive function always has to say when to stop repeating itself. There should always be two parts to a recursive function: the **recursive case** and the **base case**. The recursive case is when the function calls itself. The base case is when the function stops calling itself. This prevents infinite loops.

Recursion

Recursive functions use something called “the call stack.” When a program calls a function, that function goes on top of the call stack. This is similar to a stack of books. You add things one at a time. Then, when you are ready to take something off, you always take off the top item. Here is a recursive function to calculate the factorial of a number: `fact(3)`

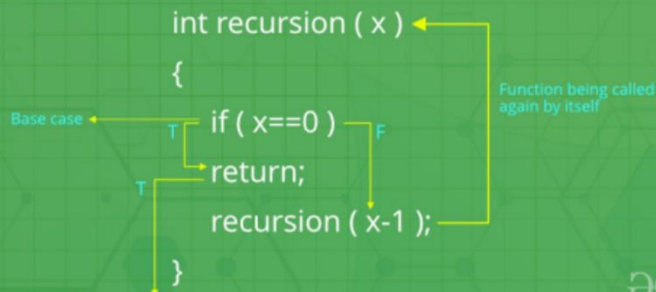
```
function fact(x) {
  if (x == 1) {
    return 1;
  } else {
    return x * fact(x-1);
  }
}
```



Recursion

```
function fact(x) {
  if (x == 1) {
    return 1;
  } else {
    return x * fact(x-1);
  }
}
```

Recursive Functions



```
x = factorial(3)
```

```
def factorial(n):
  if n == 1:
    return 1
  else:
    return n * factorial(n-1)
```

3*2 = 6
is returned

```
def factorial(n):
  if n == 1:
    return 1
  else:
    return n * factorial(n-1)
```

2*1 = 2
is returned

```
def factorial(n):
  if n == 1:
    return 1
  else:
    return n * factorial(n-1)
```

1
is returned

Recursion

Finding a Palindrome

A palindrome is a word that is spelled the same forward and backward. For example, rotor is a palindrome, but motor is not.

ROTOR vs MOTOR
 ROTOR ROTOM



So now let's say that any string containing at most one letter is a palindrome. That's our base case: a string with exactly zero letters or one letter is a palindrome.

Step 1: Establish a Base Case

Consider the word a. It's a palindrome. We can say that any string containing just one letter is by default a palindrome. Now, a string can contain no letters; we call a string of zero letters an empty string. An empty string is also a palindrome, since it "reads" the same forward and backward.

Finding a Palindrome

Step 2: Build the Recursive Case

What if the string contains two or more letters? Consider the palindrome rotor. Its first and last letters are the same, and this trait must hold for any palindrome.

On the other hand, if the first and last letters are not the same, as in motor, then the string cannot possibly be a palindrome. So now we have a way to declare that a string is not a palindrome: when its first and last letters are different. We can think of this situation as another base case, since we have the answer immediately.

Going back to when the first and last letters are the same, what does that tell us? The string might be a palindrome.

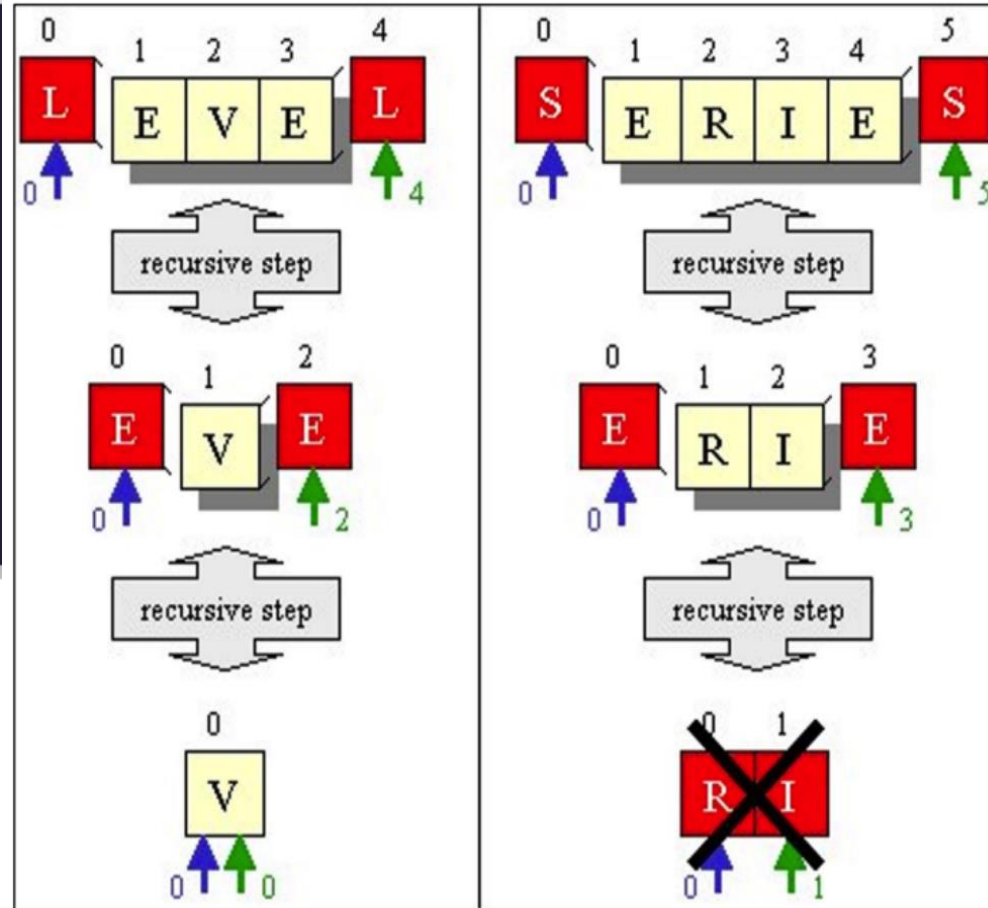
RATER vs **ROTOR**
ATE **OTO**
T
✗ ✓

Then again, it might not be. In the string rater, the first and last letters are the same, but the string is not a palindrome.

Suppose we strip off the first and last letters, leaving the string ate. Then the first and last letters of this remaining string are not the same, and so we know that rater is not a palindrome

Finding a Palindrome

```
function palindromeCheck(word) {
  if (word.length === 1 || 0) {
    return true;
  } else {
    if (word[0] === word[word.length - 1]) {
      let newWord = word.slice(1, -1);
      return palindromeCheck(newWord);
    } else {
      return false;
    }
  }
}
```



Merge Sort: A Recursive Sorting Algorithm

Merge sort follows the "divide and conquer" strategy to sort a list by repeatedly dividing it into smaller parts, sorting those parts, and then merging them to form a sorted result.

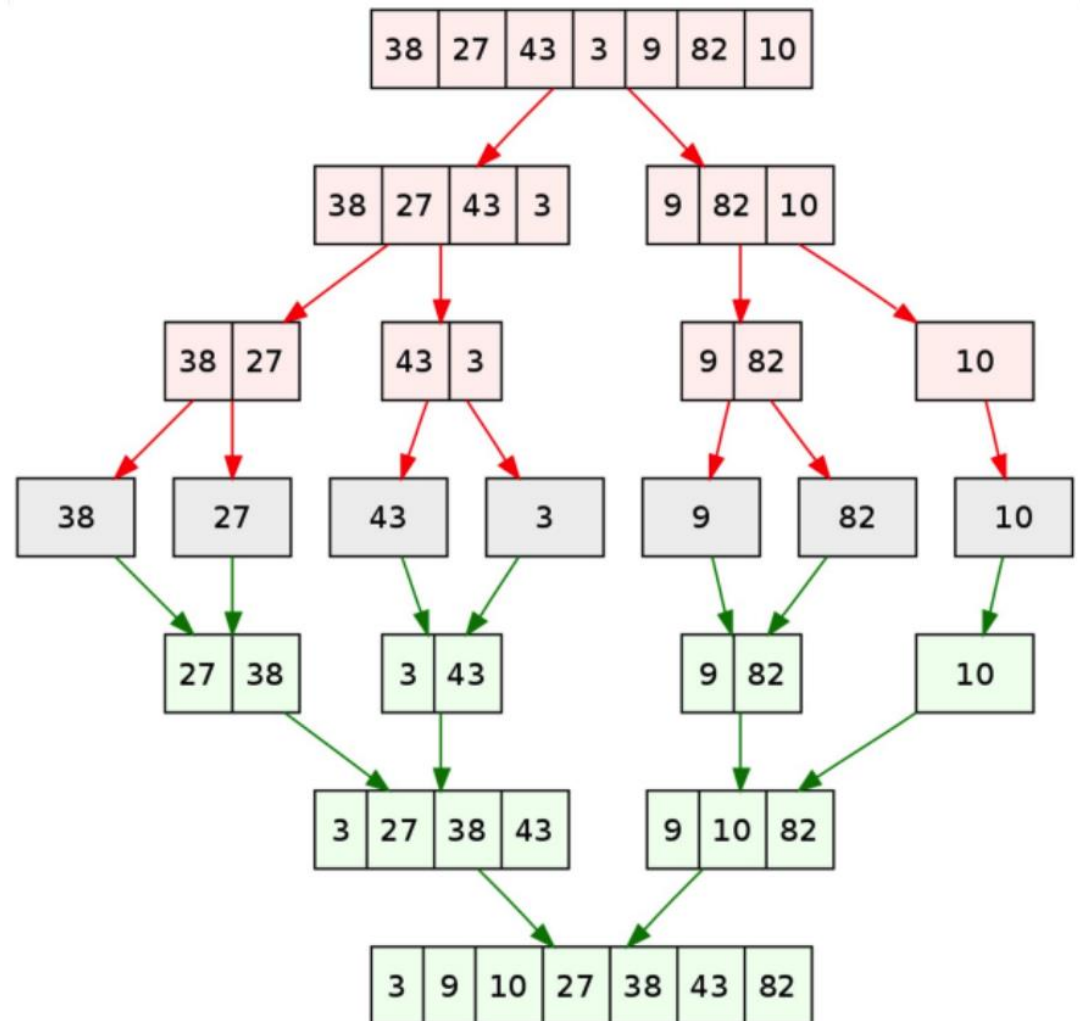
Divide: Divide the unsorted list into two equal sub lists (or nearly equal if the list has an odd number of elements). This step is recursively applied until the sub lists have only one element each.

Conquer (Sort): Sort each of the smaller sub lists (base case of recursion). For sub lists with only one element, they are already considered sorted.

Merge: Merge the sorted sub lists created in the previous step to produce a new sorted sub list. The merging process involves comparing the first elements of each sub list and placing them in order into a new merged sub list. Continue this process until all elements are merged.

Repeat: Repeat steps 1 to 3 until there's only one sorted list remaining, which is the fully sorted list.

Merge Sort: A Recursive Sorting Algorithm



Binary Tree Traversal

- Binary tree traversal is the process of visiting **every node exactly once** in a systematic order.
- Different traversal strategies produce different visiting orders, and each strategy is useful in different algorithmic situations.

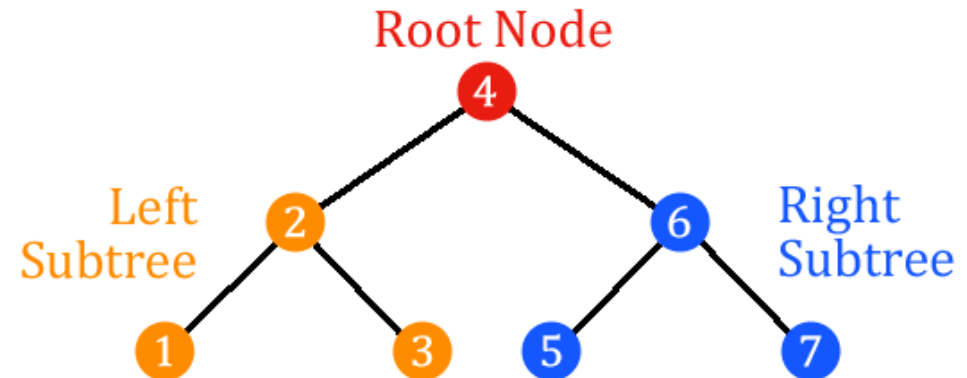
Traversal is used for:

- Searching for elements
- Printing tree contents
- Evaluating expression trees
- Finding paths
- Many graph and AI algorithms

What is a Binary Tree?

A binary tree is a hierarchical data structure in which:

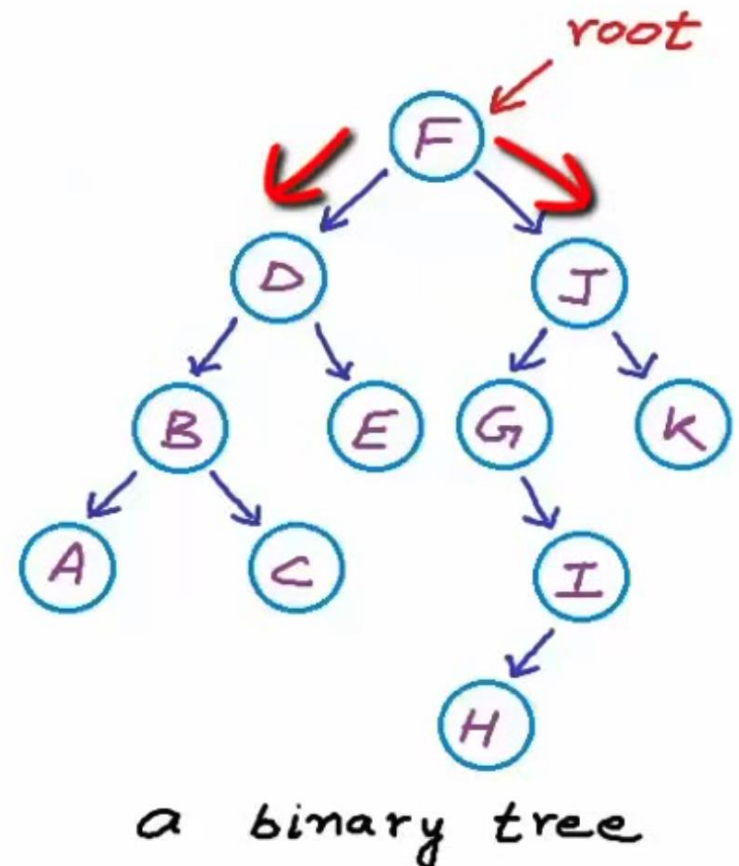
- Each node has at most two children
- The children are called:
 - Left child
 - Right child
- The top node is called the root, nodes with no children are called leaves
- Binary trees are recursive structures because each subtree is itself a binary tree.
- Visit: Reading/Processing the data of the node



Two Main Traversal Strategies

There are two main strategies for traversing a binary tree:

1. Breadth-First Search (BFS):
Also called Level-Order Traversal
 - Visits nodes level by level
 - Explores all nodes at one depth before moving deeper
2. Depth-First Search (DFS)
 - Explores as deep as possible along one branch
 - Backtracks when reaching a leaf



Breadth-First Traversal (BFS)

Visit nodes **level by level**, starting from the root.

How It Works

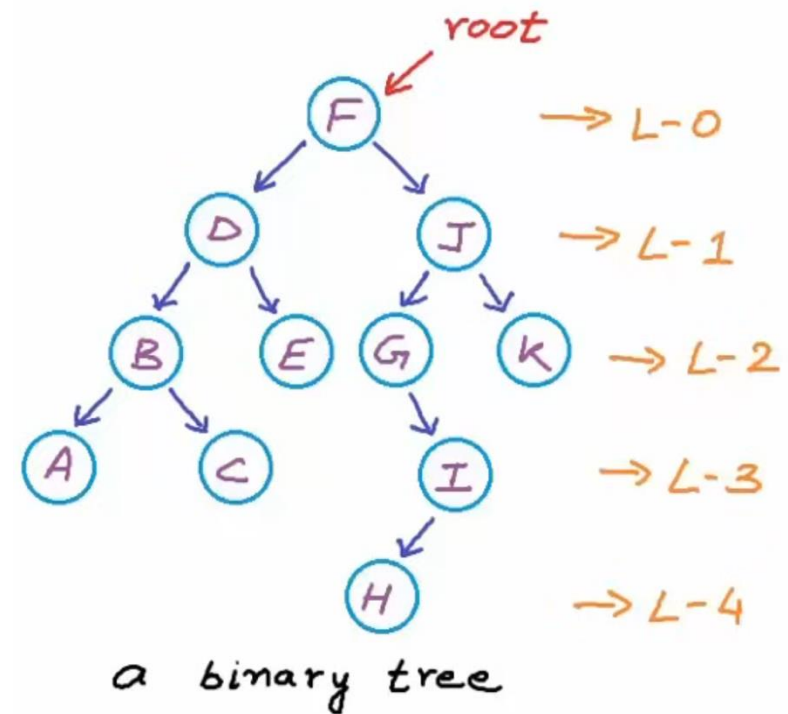
- Start at the root.
- Visit all nodes at depth 1.
- Visit all nodes at depth 2.
- Continue until all levels are visited.

Data Structure Used

Queue (FIFO: First In, First Out)

The queue ensures that nodes discovered first are processed first.

Output: F D J B E G K A C I H



Depth-First Traversal (DFS)

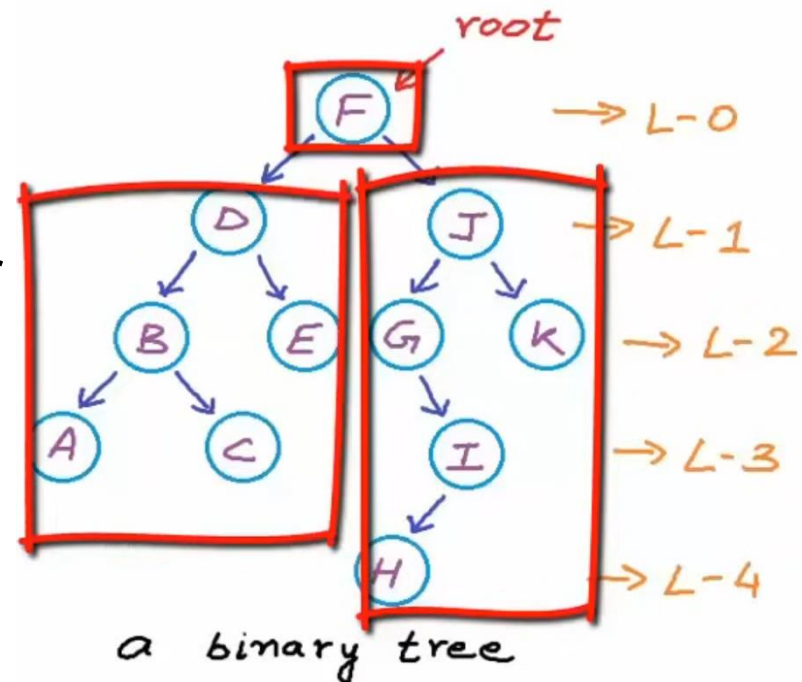
Explore one branch completely before moving to another branch.

DFS continues going deeper until:

- It reaches a leaf node
- Then it backtracks to explore other branches

Data Structure Used

- Stack (LIFO — Last In, First Out)
Or recursion (which uses an implicit stack)



Three Types of DFS

- Preorder Traversal

<Data> <Left> <Right>

Order: F D B A C E J G I H K

- Inorder Traversal

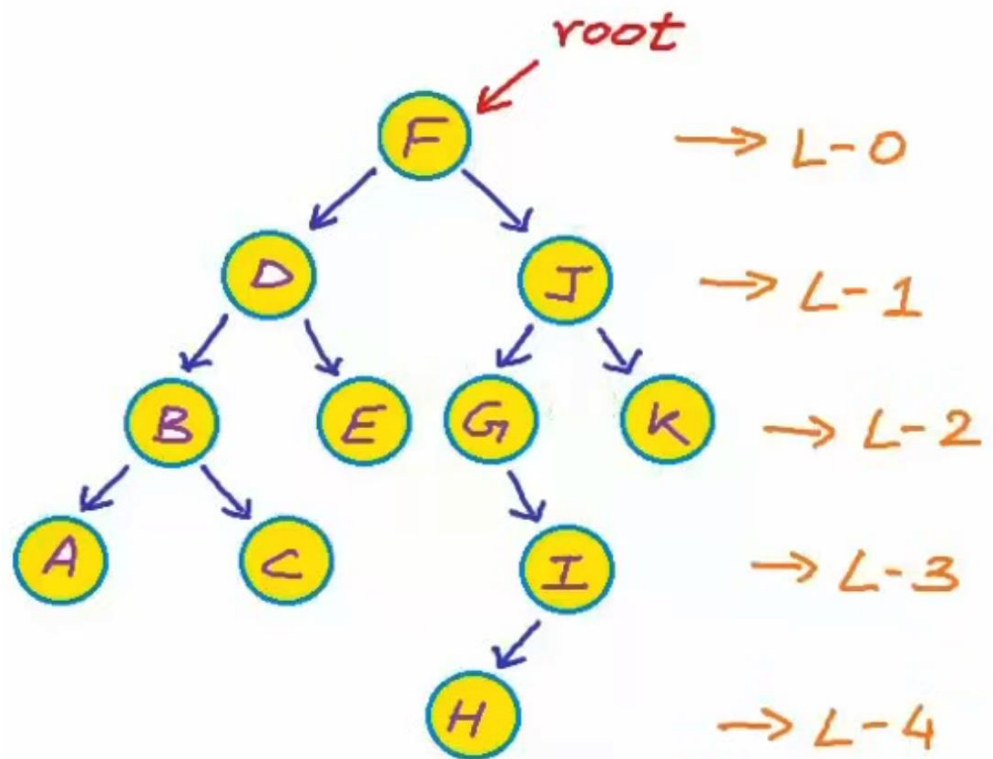
<Left> <Data> <Right>

Order: A B C D E F G H I J K

- Postorder Traversal

<Left> <Right> <Data>

Order: A C B E D H I G K J F



a binary tree

Graph Traversals

- A ***graph traversal*** starts at some vertex v and visits all the vertices on the paths that start at v . If a vertex has already been visited, it is not visited again. It follows that any traversal of a connected graph will visit all its vertices.
- A ***depth-first traversal*** starts by visiting v . Then it calls itself on each vertex adjacent to v that has not already been visited. Here's a recursive procedure to do a depth-first traversal that starts at vertex v .

```
D(v):  if  $v$  has not been visited then  
        visit( $v$ );  
        for each edge from  $v$  to  $x$  do  $D(x)$  od  
fi
```

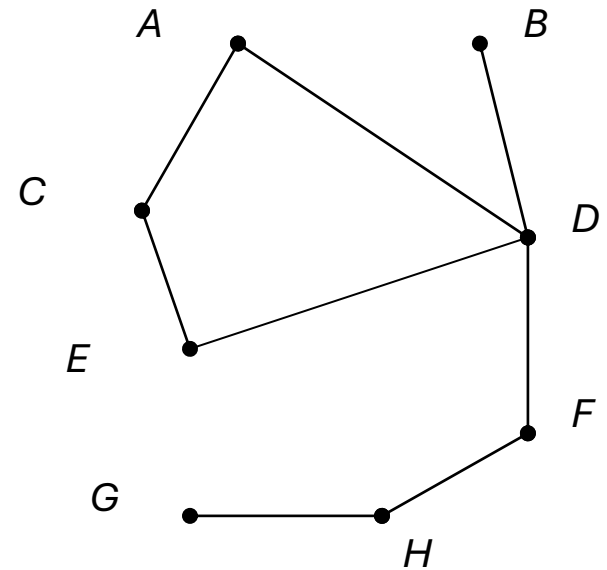
Graph Traversals

Quiz: Find a depth-first traversal of the pictured graph that starts at F .

One answer: F, H, G, D, B, A, C, E .

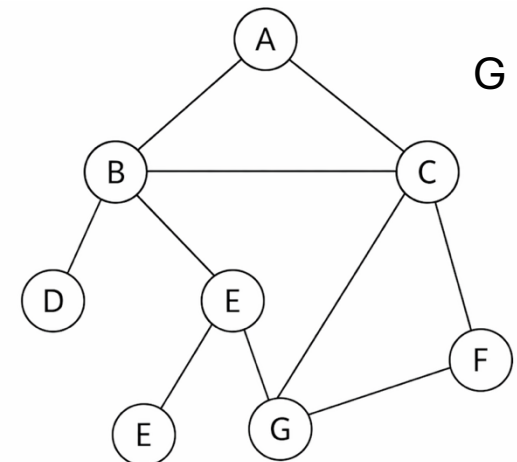
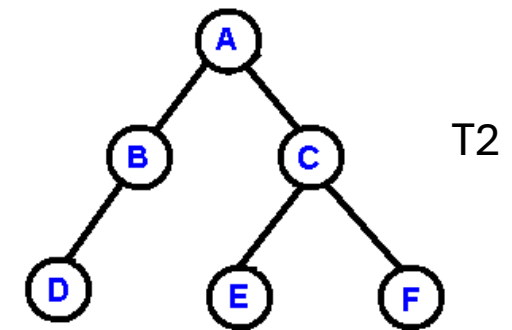
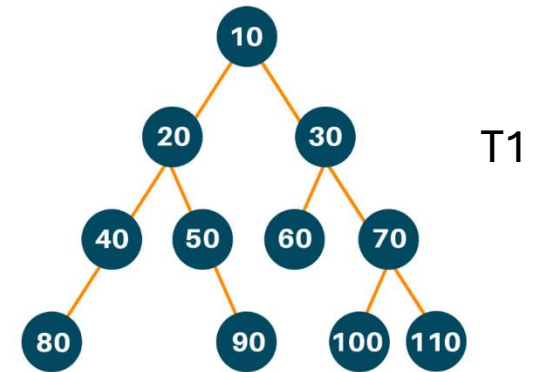
Quiz. Find a depth-first traversal of the pictured graph that starts at E .

One answer: E, D, F, H, G, A, C, B .



Binary Tree Traversal Exercise

1. Write the **BFS** traversal of T1.
2. Compute all three **DFS** traversals of T1:
Preorder (Visit, Left, Right)
Inorder (Left, Visit, Right)
Postorder (Left, Right, Visit)
3. Write the **BFS** traversal of T2.
4. Write the **DFS preorder** traversal of T2.
5. Write the **DFS inorder** traversal of T2.
6. Find a depth-first traversal of the graph G that starts at A
7. Find a depth-first traversal of the graph G that starts at E



Section 3.3 Computer Science: Grammars

A *grammar* is a finite set of rules, called *productions*, that are used to describe the strings of a language.

Notational Example. The productions take the form $\alpha \rightarrow \beta$, where α and β are strings over an alphabet of *terminals* and *nonterminals*. Read $\alpha \rightarrow \beta$ as, “ α produces β ,”

$$S \rightarrow aSB$$

$$S \rightarrow \Lambda \text{ (or } \varepsilon)$$

$$B \rightarrow bB$$

$$B \rightarrow b.$$

Alternative Short Form

$$S \rightarrow aSB \mid \Lambda \text{ (or } \varepsilon)$$

$$B \rightarrow bB \mid b.$$

Terminals: $\{a, b\}$, the alphabet of the language.

Nonterminals: $\{S, B\}$, the grammar symbols (uppercase letters), disjoint from terminals.

Start symbol: S , a specified nonterminal alone on the left side of some production.

The Language of a Grammar

The *language of a grammar* is the set of terminal strings derived from the start symbol.

Example. Can we find the language of the grammar $S \rightarrow aSB \mid L \mid \varepsilon$ and $B \rightarrow bB \mid b$?

Solution: Examine some derivations to see if a pattern emerges.

$$S \Rightarrow L$$

$$S \Rightarrow aSB \Rightarrow aB \Rightarrow ab$$

$$S \Rightarrow aSB \Rightarrow aB \Rightarrow abB \Rightarrow abbB \Rightarrow abbb$$

$$S \Rightarrow aSB \Rightarrow aaSBB \Rightarrow aaBB \Rightarrow aabB \Rightarrow aabb$$

$$S \Rightarrow aSB \Rightarrow aaSBB \Rightarrow aaBB \Rightarrow aabBB \Rightarrow aabbBB \Rightarrow aabbbbB \Rightarrow aabbbb.$$

So, we have a pretty good idea that the language of the grammar is $\{a^n b^{n+k} \mid n, k \in \mathbf{N}\}$.

Construction of Grammars

Example. Find a grammar for $\{a^n b \mid n \in \mathbf{N}\}$.

Solution: We need to derive any string of a 's followed by b .

The production $S \rightarrow aS$ can be used to derive strings of a 's. The production $S \rightarrow b$ will stop the derivation and produce the desired string ending with b .

So, a grammar for the language is $S \rightarrow aS \mid b$.

Quiz: Find a grammar for $\{ba^n \mid n \in \mathbf{N}\}$.

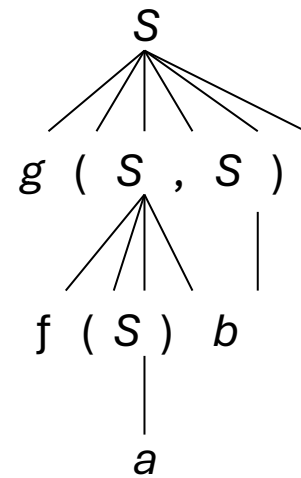
Solution: $S \rightarrow Sa \mid b$.

Parse Tree

A **Parse Tree** is a tree that represents a derivation. The root is the start symbol and the children of a nonterminal node are the symbols (terminals, nonterminals, or L) on the right side of the production used in the derivation step that replaces that node.

Example. The tree shown in the picture is the parse tree for the following derivation:

$$\begin{aligned}
 S &\Rightarrow g(S, S) \\
 &\Rightarrow g(f(S), S) \\
 &\Rightarrow g(f(a), S) \\
 &\Rightarrow g(f(a), b).
 \end{aligned}$$



Ambiguous Grammar

Means there is at least one string with two distinct parse trees, or equivalently, two distinct leftmost derivations or two distinct rightmost derivations.

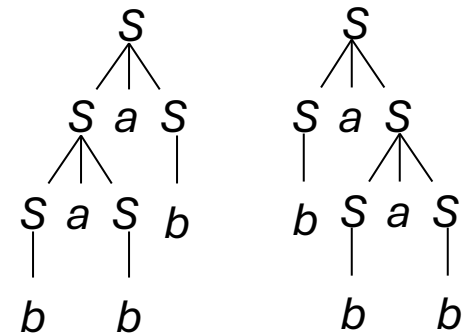
Example. Is the grammar $S \rightarrow SaS \mid b$ ambiguous?

Solution: Yes. For example, the string babab has two distinct leftmost derivations:

$S \Rightarrow SaS \Rightarrow SaSaS \Rightarrow baSaS$
 $\Rightarrow babaS \Rightarrow babab.$

$S \Rightarrow SaS \Rightarrow baS \Rightarrow baSaS \Rightarrow babaS$
 $\Rightarrow babab.$

The parse trees for the derivations are pictured.



Unambiguous Grammar:

Sometimes one can find a grammar that is not ambiguous for the language of an ambiguous grammar.

Example. The previous example showed $S \rightarrow SaS \mid b$ is ambiguous. The language of the grammar is $\{b, bab, babab, \dots\}$. Another grammar for the language is

$$S \rightarrow baS \mid b.$$

It is unambiguous because S produces either baS or b , which can't derive the same string.

Example. The previous quiz showed $S \rightarrow c \mid abS \mid Sab$ is ambiguous. Its language is $\{(ab)^m c (ab)^n \mid m, n \in \mathbf{N}\}$. Another grammar for the language is

$$S \rightarrow abS \mid cT \text{ and } T \rightarrow abT \mid L.$$

It is unambiguous because S produces either abS or cT , which can't derive the same string. Similarly, T produces either abT or L , which can't derive the same string.